

• Laravel Middleware :

هو عبارة عن طبقة (Layer) يمر عليها أي HTTP Request بمعنى يعتبره حارس امن الريكويست يفحص الطلب بتاعى واللى من خلاله بيحدد مين يدخل ومين لا وبيتحكم في الطلب بتاعى قبل وصوله الى route او controller وايضا بيتعامل مع ال response اللي بيرجلى بعد تنفيذ الطلب بتاعى

استخداماته :

- ١- بتأكد من خلاله ان اليوزر عامل تسجيل دخول ولا لا (Authentication)
- ٢- من خلاله بتحقق من صلاحية المستخدم (Authorization)
- ٣- بيوفرلى حماية CSRF
- ٤- من خلاله بيتحقق من Token
- ٥- في حالة الموقع في حالة صيانة يمنع الوصول

يتم إنشاء middleware في لارفيل من خلال الامر ده :
php artisan make:middleware MaintenanceMode

ده middleware بعمله بقوله لو موقع بتاعى في حالة صيانة وديني لصفحة maintenance

```
1 namespace App\Http\Middleware;
2
3 use Closure;
4 use Illuminate\Http\Request;
5 use Symfony\Component\HttpFoundation\Response;
6
7 class MaintenanceMode
8 {
9     /**
10      * Handle an incoming request.
11      */
12     public function handle(Request $request, Closure $next): Response
13     {
14         // حالة الصيانة
15         $maintenance = true;
16
17         if ($maintenance) {
18             return response()->view('maintenance');
19         }
20
21         return $next($request);
22     }
23 }
```

وممكن ايضا اتحكم في ال response اللي راجع انه بعد ميثاكد ويبروح لل controller بتحكم ف الرد او response اللي بيرجلى

```
$response=$next($request);

return $response;
```

في حالة لو عاوز استخدمه :

: Global Middleware -1

```
1 // app/bootstrap/app.php
2
3 return Application::configure(basePath: dirname(__DIR__))
4     ->withRouting(
5         web: __DIR__ . '/../routes/web.php',
6         commands: __DIR__ . '/../routes/console.php',
7         health: '/up',
8     )
9     ->withMiddleware(function (Middleware $middleware): void {
10         //
11         $middleware->append(
12             MaintenanceMode::class
13         );
14     })
15     ->withExceptions(function (Exceptions $exceptions): void {
16         $exceptions->shouldRenderJsonWhen(
17             fn(Request $request) => $request->is('api/*'),
18         );
19     })->create();
20
```

٢- في حالة لو عاوز اعمل middleware على route معين :

```
1 Route::get('/profile', function () {
2
3 })->middleware(MaintenanceMode::class);
4
5 // More than just middleware
6 Route::get('/profile', function () {
7
8 })->middleware([MaintenanceMode::class, Second::class]);
9
```

٣- في حالة لو هعمل group لاكثر من middleware وايضا اعمل alias لل middleware :

```
1 // app/bootstrap/app.php
2
3 ->withMiddleware(function (Middleware $middleware): void {
4     //
5     $middleware->appendToGroup('group-name', [
6         First::class,
7         Second::class
8     ]);
9
10    // middleware Alias
11    $middleware->alias([
12        "first"=> First::class,
13    ]);
14 })
15 // web.php
16 Route::get('/profile', function () {})->middleware('group-name');
17
```

ترتيب Middleware :

من خلاله بحدد الاولوية لمين بيبدأ بالترتيب

```
$middleware->priority([
    First::class,
    Second::class,
    Third::class
]);
```

وايضا يمكن ارسال بارامتر الى Middleware :

```
1 use Closure;
2 use Illuminate\Http\Request;
3
4 class CheckRole
5 {
6     public function handle(Request $request, Closure $next, string $role)
7     {
8         if (auth()->user()->role != $role) {
9             abort(403, 'ليس لديك صلاحية');
10        }
11
12        return $next($request);
13    }
14 }
15
16 // web.php
17
18 Route::get('/admin', function () {
19     return "Welcome Admin";
20 }->middleware(CheckRole::class . ':admin');
```

: Terminable Middleware

هى دالة تستخدمها في حالة بعد ال response مبرجع محتاج اعمل عملية معينة ف الخلفية بعد ارسال الرد الى المستخدم بدون متأثر او تاخر وصوله للمستخدم وليكن هسجل log او هعمل تنظيف ملفات مؤقتة

```
1 use Closure;
2 use Illuminate\Http\Request;
3 use Symfony\Component\HttpFoundation\Response;
4
5 class LogRequest
6 {
7     public function handle(Request $request, Closure $next)
8     {
9         return $next($request);
10    }
11
12    public function terminate(Request $request, Response $response)
13    {
14        \Log::info('User visited: ' . $request->path());
15    }
16 }
```

Authorization

يعنى الصلاحيات بمعنى لو عندى مستخدم معين انشى بوست وجى شخص اخر يعدل على البوست ده في الحالة لارفيلا هيرفض لان اول حاجة بيدخل يعمل Check هل الشخص ده اللي عمل البوست ايوه ابقى اسمحله يعدل على البوست غير كده لا وده اسمه الصلاحيات او Authorization وتوفرلى طريقتين :

١- Gates :

وهى عبارة عن دالة Closure من خلالها بتتحق من صلاحية معينة وبيتم تعريفها في AppServiceProvider ويستخدم في الصلاحيات العامة وليكن هل مسموحه يفتح dashboard

```
1 Gate::define('update-post', function (User $user, Post $post) {
2     return $user->id === $post->user_id;
3 });
```

وبعدها استخدم Gate في controller

```
1 if (Gate::allows('update-post', $post)) {
2     return true
3 }
4 // or
5
6 Gate::authorize('update-post', $post);
7
8
```

إذا لم يكن لديه صلاحية Laravel يرمي Exception ويحولها تلقائياً إلى 403 forbidden

وفي حالة عاوز افحص مستخدم معين

```
1 Gate::forUser($user)->allows('update-post', $post);
```

وايضا في لو اليوزر عنده اى صلاحية من مجموعة من الصلاحيات ده

```
1 Gate::any([
2     'update-post',
3     'delete-post'
4 ], $post);
```

وايضا يمكن ارجاع response برسالة معينة بدل من true
return Response::allow(); or return Response::deny("You are not admin");

وايضا يمكن تغيير كود الخطا بدل مكان بيرجع ٤٠٣ اخلية يرجع ايورر معين او صفحة غير موجودة ٤٠٤
Response::denyWithStatus(404); Response::denyAsNotFound();

وايضا يمكن عمل Inline Authorization بدل من عمل Gate كاملة :

```
Gate::allowIf(fn(User $user)=>$user->isAdmin());
```

٢- Policies :

هو عبارة عن class يستخدم في الصلاحيات التي ترتبط ب model معين وليكن حذف او تعديل بوست معين
ويتأكد انك مين اللي عمل البوست عشان يقدر يحذف او

بيتم انشاء polices باستخدام الامر ده وطبعاً بيتم ربط بال model :

```
php artisan make:policy PostPolicy --model=Post
```

```

1
2  use App\Models\Post;
3  use App\Models\User;
4
5  class PostPolicy
6  {
7      public function update(User $user, Post $post)
8      {
9          return $user->id === $post->user_id;
10     }
11 }
12

```

ويتم استخدام الـ Policy داخل Controller

```

1  public function edit(Post $post)
2  {
3      $this->authorize('update', $post);
4
5      return view('posts.edit', compact('post'));
6  }

```

ويتم استخدام Gate في blade :

```
1 @can('update', $post)
2   <a href="{{ route('posts.edit', $post) }}">Edit</a>
3 @endcan
```

استخدام authorizeResource :
لو عندي Resource Controller بيتتم ربطه ب policy مرة واحدة

```
1 @can('update', $post)
2   <a href="{{ route('posts.edit', $post) }}">Edit</a>
3 @endcan
```